



SESSION 8

RESTFUL AND HTTP APIS

Mục tiêu học tập

Trong buổi học này, học sinh sẽ học cách:

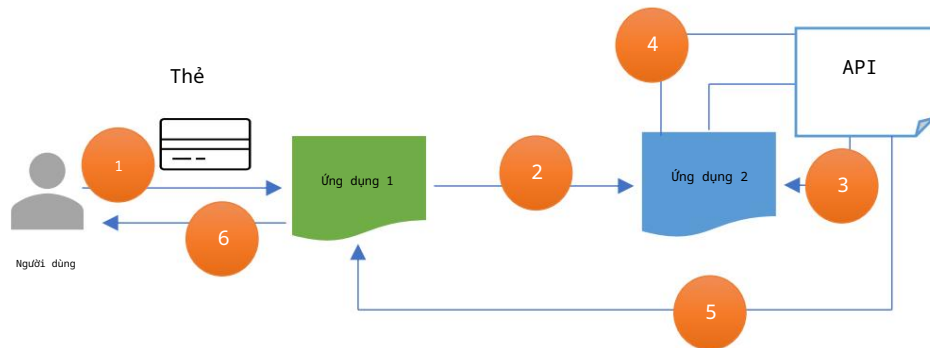
- Mô tả API và mục đích của nó
- Liệt kê các loại API phổ biến trong Node.js và chức năng của chúng
- Giải thích API RESTful và HTTP và các tính năng của chúng
- Giải thích cách sử dụng RESTful API và HTTP API để quản lý dữ liệu
- Xác định ưu điểm và nhược điểm của API RESTful và HTTP
- Phân biệt giữa API RESTful và API HTTP

Giao diện lập trình ứng dụng (API) dịch hướng dẫn của người dùng sang định dạng mà hệ thống có thể diễn giải để xử lý yêu cầu và gửi lại phản hồi.

Ví dụ, một ứng dụng thương mại điện tử (ví dụ: App1) cung cấp tùy chọn thanh toán bằng thẻ tín dụng. Để đảm bảo giao dịch thanh toán diễn ra suôn sẻ, App1 phải giao tiếp với ứng dụng của công ty thẻ tín dụng (ví dụ: App2) để xử lý xác thực và xử lý các khoản thanh toán được thực hiện bằng thẻ tín dụng.

Mỗi công ty thẻ tín dụng sẽ có ứng dụng xử lý thanh toán riêng. Để giao tiếp với App2, APP1 phải sử dụng API. Công ty thẻ tín dụng thường cung cấp API này. API này sẽ đóng vai trò trung gian giữa App1 và App2 để

hoàn tất giao dịch cho người dùng.



Hình 8.1: Hệ thống xử lý thanh toán

Hình 8.1 minh họa quy trình làm việc của hệ thống xử lý thanh toán. Trong trường hợp này:

1. Người dùng nhập thông tin thẻ vào trang web thanh toán.
2. App1 gửi dữ liệu đến App2.
3. Tại App2, các chi tiết được API tiếp nhận, sau đó API sẽ dịch dữ liệu sang định dạng mà App2 có thể hiểu được.
4. Sau khi thanh toán được xử lý, API sẽ lấy trạng thái từ App2 và dịch nó sang ngôn ngữ mà App1 có thể hiểu được.
5. Sau đó, App2 gửi trạng thái thanh toán đến App1.
6. App1 sau đó hiển thị trạng thái thanh toán trên trang Web cho

người dùng.

Phiên này giới thiệu các loại API phổ biến trong Node.js và ứng dụng của chúng.

Tài liệu này cũng liệt kê các tính năng của API Chuyển giao trạng thái biểu diễn (RESTful) và API Giao thức truyền siêu văn bản (HTTP) cũng như các lợi ích và hạn chế của chúng.

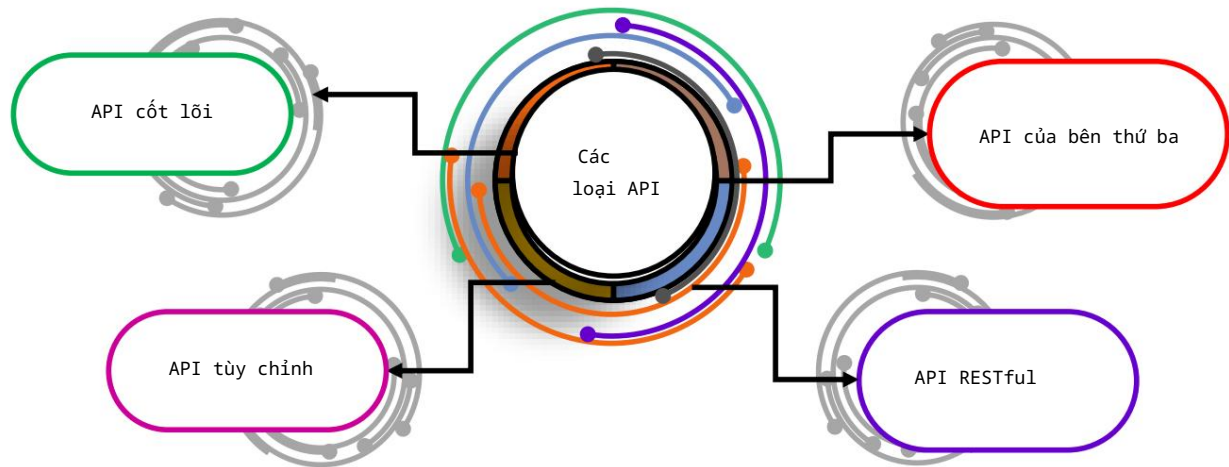
Cuối cùng, bài viết giải thích các ứng dụng của API RESTful và HTTP cũng như các tính năng khác biệt giữa API RESTful và HTTP.

8.1 Giới thiệu về API

Như đã thảo luận, API tạo điều kiện thuận lợi cho việc giao tiếp và tương tác giữa nhiều ứng dụng phần mềm khác nhau thông qua một bộ quy tắc và giao thức.

Mục tiêu chính của API là sắp xếp các yêu cầu và phản hồi để cho phép các nhà phát triển trích xuất dữ liệu từ các ứng dụng phần mềm khác.

Tùy thuộc vào phạm vi và cách sử dụng, các API được Node.js hỗ trợ như sau:



- **API lõi:** Các API này được cung cấp bởi Node.js và có sẵn dưới dạng các mô-đun tích hợp, chẳng hạn như mô-đun hệ thống tệp hoặc mô-đun HTTP. Các API lõi cung cấp các khối nền tảng cơ bản của một ứng dụng. Ví dụ, hãy xem xét một Người dùng muốn xây dựng một trang web thương mại điện tử. Tại đây, các API cốt lõi sẽ cung cấp các chức năng cơ bản cho phép khách hàng duyệt danh mục sản phẩm, quản lý giỏ hàng, quản lý hàng tồn kho, quản lý đơn hàng và quản lý thanh toán.
- **API của bên thứ ba:** Đúng như tên gọi, các API này được cung cấp bởi các nhà phát triển hoặc công ty khác, cho phép người dùng tích hợp vào ứng dụng của họ để cung cấp các tính năng chuyên biệt. Các API này được cài đặt bằng Node Package Manager (npm). Một số ví dụ về API của bên thứ ba bao gồm mongoose để kết nối với cơ sở dữ liệu MongoDB, Express.js để sử dụng các framework web và Passport.js để xử lý xác thực. Trong ứng dụng thương mại điện tử, người dùng có thể sử dụng API của bên thứ ba để cung cấp tính năng đánh giá sản phẩm. Khách hàng có thể sử dụng tính năng này để cung cấp đánh giá sản phẩm và xem các đánh giá do khách hàng khác thêm vào.
- **API tùy chỉnh:** Các API này được sử dụng để phát triển chức năng mới mà ứng dụng yêu cầu. Người dùng có thể sử dụng đối tượng `module.exports` để tạo chức năng và cung cấp cho các phần khác của ứng dụng. Trong ứng dụng thương mại điện tử, API tùy chỉnh có thể được sử dụng để cung cấp cho khách hàng các đề xuất được cá nhân hóa dựa trên lịch sử duyệt web và mua sắm của họ.
- **API RESTful:** Đúng như tên gọi, các API này hoạt động dựa trên các nguyên tắc của kiến trúc Chuyển giao Trạng thái Biểu diễn (REST). Chúng là loại API HTTP sử dụng các yêu cầu HTTP để truy cập và quản lý dữ liệu. Các API này xác định cách dữ liệu được chia sẻ giữa các phần khác nhau của ứng dụng. Trong ứng dụng thương mại điện tử, API RESTful có thể được sử dụng để đảm bảo dữ liệu được truyền theo định dạng chuẩn hóa như JSON hoặc CML.

Nhà phát triển có thể chọn API cụ thể dựa trên yêu cầu của ứng dụng.

8.2 API HTTP

Node.js hỗ trợ API HTTP, sử dụng giao thức HTTP để tạo điều kiện thuận lợi cho việc giao tiếp và trao đổi dữ liệu giữa các hệ thống. Các API này sử dụng các phương thức HTTP, chẳng hạn như GET, POST, PUT và DELETE, để gửi yêu cầu đến máy chủ. Chúng sử dụng các định dạng phổ biến, chẳng hạn như JSON và XML, để gửi phản hồi về máy khách.

Sau đây là một số tính năng của API HTTP:

Máy khách-máy chủ Người mẫu	Họ duy trì sự phân biệt rõ ràng giữa máy khách và máy chủ. Trong mô hình, khách hàng có trách nhiệm yêu cầu hành động hoặc dữ liệu và máy chủ chịu trách nhiệm xử lý và phản hồi yêu cầu.
nhiều lớp Ngành kiến trúc	Họ cho phép nhiều hệ thống trung gian tham gia vào giao tiếp giữa máy khách và máy chủ. Kiến trúc này cung cấp tính linh hoạt bằng cách hỗ trợ phân phối trách nhiệm xử lý giữa các hệ thống khác nhau.
Bộ nhớ đệm	Chúng cho phép các hệ thống trung gian, chẳng hạn như máy chủ Web và proxy máy chủ, để lưu trữ các phản hồi cho dữ liệu được yêu cầu thường xuyên. Điều này tính năng giúp cải thiện hiệu suất của ứng dụng bằng cách giảm tải cho máy chủ.
Đồng phục Giao diện	Họ sử dụng một giao diện nhất quán dựa trên các phương thức HTTP phổ biến và Mã định danh tài nguyên thống nhất (URI) chuẩn để làm việc với dữ liệu. Giao diện thống nhất này giúp các nhà phát triển dễ dàng phát triển và duy trì các ứng dụng API RESTful.
Tiêu chuẩn Các loại phương tiện truyền thông	Họ sử dụng các loại phương tiện truyền thông tiêu chuẩn, chẳng hạn như JSON và XML, cho trao đổi dữ liệu giữa máy khách và máy chủ. Tính năng này làm cho Các ứng dụng API HTTP có thể tương tác và tương thích trên nhiều nền tảng khác nhau.
Lấy tài nguyên làm trung tâm Tiếp cận	Họ coi dữ liệu như tài nguyên. Các tài nguyên này được xác định bằng URI và có thể được thao tác bằng các phương thức HTTP. Tính năng này làm cho Cấu trúc API dễ hiểu và dễ triển khai.
Không quốc tịch Tương tác	Trong các API này, máy chủ không lưu giữ bất kỳ thông tin nào liên quan đến trạng thái cụ thể của khách hàng. Điều này có nghĩa là mỗi yêu cầu chứa tất cả thông tin cần thiết để hiểu và xử lý yêu cầu, mà không cần bất kỳ sự phụ thuộc nào vào yêu cầu trước đó hoặc tiếp theo.

Các phương pháp phổ biến được API HTTP sử dụng như sau:

LẤY	BUU KIỆN	ĐẶT	XÓA BỎ	MIẾNG VÁ
Được sử dụng để gửi yêu cầu truy xuất dữ liệu từ máy chủ	Được sử dụng để gửi yêu cầu có thông tin cần thiết đến máy chủ để tạo một yêu cầu mới tài nguyên	Được sử dụng để gửi yêu cầu có thông tin cần thiết đến máy chủ để cập nhật thông tin hiện có tài nguyên	Được sử dụng để gửi yêu cầu đến máy chủ để xóa một tài nguyên	Được sử dụng để gửi yêu cầu đến máy chủ để cập nhật một phần tài nguyên

8.3 Ưu điểm và nhược điểm của API HTTP

Bảng 8.1 liệt kê một số ưu điểm và nhược điểm của API HTTP.

Thuận lợi		Nhược điểm	
Sự đơn giản	Các API này dễ hiểu, dễ thiết kế, dễ triển khai và dễ bảo trì. Điều này là do chúng sử dụng các phương thức HTTP tiêu chuẩn và định dạng dữ liệu chung.	Độ phức tạp	Các API này không phù hợp để làm việc với các truy vấn phức tạp hoặc tổng hợp dữ liệu đòi hỏi phải xử lý nhiều yêu cầu cùng lúc.
Khả năng mở rộng	Các API này có khả năng mở rộng nhờ tính chất không trạng thái và kiến trúc hướng tài nguyên. Tính năng này cho phép các API này thích ứng với các yêu cầu thay đổi mà không cần thay đổi đáng kể về kiến trúc.	Hiệu suất Giới hạn	Các API này hoạt động theo chu trình yêu cầu-phản hồi và sử dụng các phương thức HTTP ít biểu đạt hơn. Tính năng này đặt ra giới hạn về chức năng và độ phức tạp của ứng dụng. Do đó, các API này không phù hợp với các ứng dụng yêu cầu phát trực tuyến theo thời gian thực hoặc giao dịch tần suất cao.

Thuận lợi		Nhược điểm	
Hiệu quả	Các API này cung cấp hiệu suất hiệu quả vì phản hồi cho các yêu cầu thường xuyên có thể được lưu vào bộ nhớ đệm, do đó giảm tải cho máy chủ và cung cấp phản hồi nhanh chóng cho các yêu cầu đó.	Bảo vệ Mối quan tâm-	Các API này phụ thuộc vào giao thức HTTP để xác thực và mã hóa dữ liệu. Tuy nhiên, các dịch vụ do HTTP cung cấp lại yếu và dễ bị tấn công và xâm phạm.
Tính linh hoạt	Các API này có thể dễ dàng thích ứng với nhiều yêu cầu khác nhau. Chúng cũng hỗ trợ nhiều nền tảng và ngôn ngữ.	Giới hạn Lỗi Xử lý	Các API này sử dụng mã trạng thái HTTP để truyền tải lỗi. Các mã này không cung cấp đủ thông tin để có thể xác định lỗi và khắc phục sự cố. Vì vậy, các nhà phát triển cần triển khai thêm các cơ chế gỡ lỗi.
Tính di động	Các API này không phụ thuộc vào nền tảng vì chúng sử dụng giao thức HTTP chuẩn và định dạng dữ liệu chung.	Giới hạn Chuẩn hóa	Các API này không có bất kỳ thông số kỹ thuật hoặc xác thực chính thức nào. Điều này có thể dẫn đến sự không nhất quán và mơ hồ giữa các triển khai khác nhau.

Bảng 8.1: Ưu điểm và nhược điểm của API HTTP

8.4 API RESTful

API RESTful là một API Web sử dụng giao thức HTTP để trao đổi dữ liệu giữa các ứng dụng. Đây là một tập hợp con của API HTTP, tuân theo các nguyên tắc và hạn chế của kiến trúc REST. API này được sử dụng để thiết kế các ứng dụng mạng, đặc biệt là các dịch vụ Web. Tương tự như API HTTP, API RESTful sử dụng các phương thức HTTP, chẳng hạn như GET, POST, PUT và DELETE, để thực hiện các thao tác trên tài nguyên.

API RESTful cung cấp các tính năng, ưu điểm và nhược điểm giống như API HTTP.

8.5 API RESTful so với API HTTP

API RESTful và HTTP có nhiều tính năng, ưu điểm và nhược điểm chung. Tuy nhiên, cả hai đều khác nhau ở một số khía cạnh, chẳng hạn như nguyên tắc và tiêu chuẩn thiết kế.

Bảng 8.2 liệt kê những điểm khác biệt giữa hai API này.

API RESTful		API HTTP
Phong cách kiến trúc	API RESTful tuân theo các giới hạn kiến trúc và nguyên tắc thiết kế cụ thể của REST, bao gồm chiến lược dựa trên tài nguyên, phương thức yêu cầu HTTP chuẩn hóa, giao tiếp không trạng thái giữa máy khách và máy chủ và giao diện thống nhất.	API HTTP không tuân theo các nguyên tắc thiết kế REST. Nó linh hoạt để phù hợp với yêu cầu của ứng dụng, xác định điểm cuối và sử dụng các phương thức HTTP mà không gặp nhiều hạn chế.
Mẫu URL	API RESTful dựa trên tài nguyên, trong đó mỗi URL đại diện cho một tài nguyên hoặc tập hợp các tài nguyên. URL bao gồm một mã định danh tài nguyên (ID) để chỉ định tài nguyên cần được truy cập hoặc thao tác.	API HTTP linh hoạt hơn, trong đó các điểm cuối không hoàn toàn dựa trên tài nguyên và bao gồm các mã định danh tài nguyên có thể được cấu trúc tùy thuộc vào thiết kế của API.
Tiêu chuẩn hóa thiết kế	Tất cả các API RESTful đều có mẫu thiết kế chuẩn hóa và phong cách kiến trúc cố định.	API HTTP không có cùng mức độ chuẩn hóa và có sự khác biệt đáng kể về kiểu thiết kế và phong cách kiến trúc.
Có thể quản lý Độ chi tiết	API RESTful áp dụng tính chi tiết trong mã giúp các nhà phát triển dễ dàng phát triển và bảo trì ứng dụng.	API HTTP không áp dụng tính chi tiết trong mã, khiến cho việc này trở nên khó khăn

API RESTful		API HTTP
		để phát triển các ứng dụng phức tạp.
Phiên bản	API RESTful thực thi việc quản lý phiên bản.	API HTTP không áp dụng quản lý phiên bản.
Siêu phương tiện như Động cơ của Trạng thái ứng dụng (GHÉT)	Trong API RESTful, HATEOAS là một nguyên tắc thiết kế. Nguyên tắc này nhấn mạnh việc sử dụng các liên kết siêu phương tiện trong phản hồi để hướng dẫn người dùng qua API và khám phá các tài nguyên và hành động khả dụng.	Trong API HTTP, HATEOAS là một thành phần tùy chọn được triển khai trong các ứng dụng cụ thể khi cần thiết.

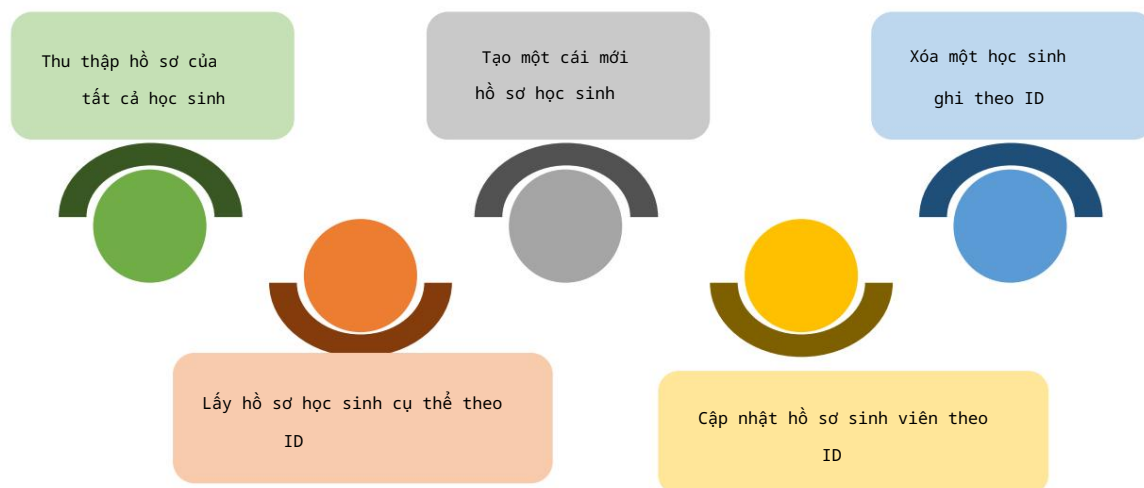
Bảng 8.2: Sự khác biệt giữa API RESTful và API HTTP

8.6 Sử dụng API HTTP với Express.js

Express.js là một nền tảng ứng dụng web Node.js mã nguồn mở dùng để tạo trang web và API. Nền tảng này xử lý các yêu cầu HTTP khác nhau tùy theo yêu cầu.

Giả sử một người dùng muốn tạo một API HTTP đơn giản để quản lý dữ liệu của sinh viên trong ứng dụng Node.js bằng cách sử dụng framework Express.js.

Ứng dụng này sẽ cho phép người dùng thực hiện các tác vụ sau:



Để chạy ứng dụng, người dùng muốn sử dụng cURL làm máy khách để tương tác với máy chủ. cURL là một công cụ có thể được sử dụng để gửi yêu cầu đến máy chủ và nhận phản hồi từ máy chủ. Để sử dụng cURL, người dùng phải có giao diện dòng lệnh, trong trường hợp này là Node.js, và kết nối Internet.

Để tạo ứng dụng này, hãy thực hiện các bước sau:

1. Đảm bảo rằng Node.js và Express.js đã được cài đặt.
2. Mở VS Code hoặc tệp Notepad.

- Để nhập khung Express.js, hãy tạo một phiên bản của ứng dụng Express.js và chỉ định cổng sẽ được sử dụng cho ứng dụng Web.

Sau đó, trong tệp, nhập mã được cung cấp trong Đoạn mã 1.

Đoạn mã 1:

```
const express = require('express');  
ứng dụng const = express();  
cổng const = 3000;  
  
ứng dụng.use(express.json());
```

Trong Đoạn mã 1, hàm require được sử dụng để import framework Express.js. Sau đó, một thể hiện của ứng dụng Express được tạo và lưu trữ trong biến app. Cuối cùng, cổng được sử dụng cho ứng dụng này được chỉ định là 3000.

- Để thêm một số dữ liệu mẫu về học sinh, trong cùng một tệp, hãy thêm mã được đưa ra trong Đoạn mã 2.

Đoạn mã 2:

```
// Dữ liệu mẫu cho sinh viên  
cho học sinh = [  
  { id: 1, tên: 'David Miller', tuổi: 12, điểm: 'A' },  
  { id: 2, tên: 'Adam Smith', tuổi: 13, điểm: 'B' },  
  { id: 3, tên: 'John Williams', tuổi: 12, điểm: 'A' },  
];  
hãy để nextStudentId = 4;
```

Trong Đoạn mã 2, một mảng có tên students được tạo để lưu trữ dữ liệu mẫu của các học sinh. Mỗi học sinh đại diện cho một đối tượng được cấu hình với các thuộc tính ID, name, age và grade. Biến nextStudentId

khởi tạo một giá trị để gán ID duy nhất cho sinh viên mới. Trong đoạn mã này, biến này được khởi tạo thành 4 vì ba sinh viên đầu tiên đã có mặt trong mảng.

- Để xem thông tin chi tiết của tất cả học sinh, trong cùng một tệp, hãy thêm mã được đưa ra trong Đoạn mã 3.

Đoạn mã 3:

```
// Lấy tất cả học sinh  
ứng dụng.get('/sinh viên', (yêu cầu, res) => {  
  res.json(học sinh);  
});
```

Mã trong Đoạn mã 3 gửi thông tin chi tiết về học sinh theo định dạng JSON khi điểm cuối của yêu cầu là /students.

- Để xem thông tin chi tiết của một học sinh có ID cụ thể, trong cùng một tệp, thêm mã được đưa ra trong Đoạn mã 4.

Đoạn mã 4:

```
// Lấy một học sinh cụ thể theo ID
ứng dụng.get('/sinh viên/:id', (yêu cầu, res) => {
  const id = ParseInt(req.params.id);
  const student = students.find((s) => s.id === id);
  nếu (học sinh) {
    res.json(sinh viên);
  } khác {
    res.status(404).json({ message: 'Không tìm thấy học sinh'
  });
}
});
```

Đoạn mã 4 lưu trữ ID nhận được từ máy khách trong biến id.

Sau đó, nó tìm kiếm học sinh có ID được lưu trữ trong biến id và lưu trữ thông tin chi tiết về học sinh đó trong biến student. Cuối cùng, nó sử dụng khối if-else để trả về thông tin học sinh nếu học sinh đó tồn tại hoặc trả về thông báo lỗi nếu học sinh đó không tồn tại.

- Để thêm thông tin chi tiết về một học sinh mới, trong cùng một tệp, hãy thêm mã được cung cấp trong Đoạn mã 5.

Đoạn mã 5:

```
// Tạo một sinh viên mới
ứng dụng.post('/sinh viên', (yêu cầu, res) => {
  const { tên, tuổi, lớp } = req.body;
  const student = { id: nextStudentId++, tên, tuổi, lớp };

  students.push(học sinh);
  res.status(201).json(học sinh);
});
```

Đoạn mã 5, đầu tiên phân tích cú pháp và lưu trữ thông tin học sinh nhận được từ máy khách vào một mảng. Sau đó, nó tạo một học sinh bằng cách sử dụng thông tin trong mảng và ID được lưu trữ trong biến nextStudentId, rồi tăng giá trị của biến này. Cuối cùng, nó gửi thông tin học sinh mới đến máy khách ở định dạng JSON.

- Để cập nhật thông tin chi tiết của một học sinh có ID cụ thể, trong cùng một tệp, thêm mã được đưa ra trong Đoạn mã 6.

Đoạn mã 6:

```
// Cập nhật học sinh theo ID

ứng dụng.put('/sinh viên/:id', (yêu cầu, res) => {
  const id = ParseInt(req.params.id);
  const student = students.find((s) => s.id === id);

  nếu (!học sinh) {

    res.status(404).json({ message: 'Không tìm thấy học sinh'
  });

  } khác
  {
    const { tên, tuổi, lớp } = req.body;
    tên sinh viên = tên;
    student.age = tuổi;
    student.grade = điểm;
    res.json(sinh viên);
  }
});
```

Đoạn mã 6 phân tích cú pháp ID nhận được từ máy khách và lưu trữ nó trong id Biến. Sau đó, nó sử dụng ID trong biến id để tìm kiếm học sinh. Nó lưu trữ thông tin chi tiết về học sinh được tìm thấy trong biến student. Sau đó, nó kiểm tra xem biến student có tồn tại hay không. Nếu không tồn tại, một thông báo lỗi sẽ được gửi lại cho máy khách. Nếu biến student tồn tại, thông tin học sinh sẽ được thay thế bằng thông tin nhận được trong yêu cầu của máy khách. Thông tin học sinh được cập nhật này sau đó sẽ được gửi lại cho máy khách ở định dạng JSON.

9. Để xóa thông tin chi tiết của một học sinh có ID cụ thể trong cùng một tệp, thêm mã được đưa ra trong Đoạn mã 7.

Đoạn mã 7:

```
// Xóa học sinh theo ID
ứng dụng.xóa('/sinh viên/:id', (yêu cầu, res) => {
  const id = ParseInt(req.params.id);
  const index = students.findIndex((s) => s.id === id);
  nếu (chỉ số !== -1) {
    students.splice(chỉ số, 1);
    res.json({ message: 'Sinh viên đã bị xóa' });
  } khác {
    res.status(404).json({ message: 'Không tìm thấy học sinh'
  });
  }
});
```

Đoạn mã 7 phân tích cú pháp ID nhận được từ máy khách và lưu trữ trong biến id. Sau đó, nó sử dụng ID trong biến id để tìm kiếm chỉ mục của học sinh. Nó lưu trữ chỉ mục của học sinh được tìm thấy trong biến index. Sau đó, nó kiểm tra xem biến index có giá trị khác -1 hay không. Nếu điều kiện được đáp ứng, phương thức splice được sử dụng để xóa thông tin học sinh tại chỉ mục được chỉ định. Thông tin học sinh được cập nhật sau đó được gửi lại cho máy khách ở định dạng JSON. Nếu điều kiện không được đáp ứng, một thông báo lỗi sẽ được gửi lại cho máy khách.

10. Để khởi động máy chủ và cấu hình để lắng nghe cổng 3000, trong cùng một tệp, hãy thêm mã được đưa ra trong Đoạn mã 8.

Đoạn mã 8:

```
// Khởi động máy chủ
ứng dụng.listen(cổng, () => {
  console.log(`Máy chủ đang chạy trên cổng ${port}`);
});
```

11. Lưu tệp dưới dạng studentHTTApi.js trong cùng thư mục với Express.js đã được cài đặt.

Bảng 8.3 liệt kê các phương thức HTTP chuẩn và các điểm cuối được xác định trong mã.

Hoạt động	HTTP Phương pháp	Điểm cuối
Để lấy lại tất cả hồ sơ học sinh	GET	app.get('/students', (req, res))
Để lấy hồ sơ học sinh dựa trên ID	NHẬN	ứng dụng.get('/students/:id', res) (yêu cầu,
Để thêm hồ sơ sinh viên mới vào cơ sở dữ liệu	POST	app.post('/students', (req, res))
Để cập nhật thông tin chi tiết của học sinh dựa trên ID	ĐẶT	ứng dụng.put('/students/:id', (req, res))
Để xóa hồ sơ học sinh dựa trên ID	XÓA	ứng dụng.delete('/students/:id', (req, res))

Bảng 8.3: Phương thức HTTP và Điểm cuối được Xác định cho Ứng dụng

12. Để khởi động máy chủ, hãy mở cửa sổ Dấu nhắc lệnh.

13. Tại Dấu nhắc lệnh, hãy chạy lệnh như sau:

```
nút studentHTTApi.js
```

Lệnh này khởi động máy chủ. Khi máy chủ đã hoạt động, một thông báo sẽ được ghi vào bảng điều khiển cho biết máy chủ đang lắng nghe trên cổng được chỉ định, như minh họa trong Hình 8.2.

```
C:\MyApps>node studentHTTApi.js  
Server is running on port 3000
```

Hình 8.2: Khởi động máy chủ

Máy chủ hiện đã sẵn sàng để nhận yêu cầu và gửi phản hồi.

14. Để sử dụng curl để tương tác với máy chủ Web, hãy mở một cửa sổ Dấu nhắc lệnh khác.
15. Để lấy hồ sơ của tất cả học sinh, tại dấu nhắc lệnh, hãy chạy lệnh như sau:

```
curl http://localhost:3000/students
```

Lệnh này hiển thị thông tin về tất cả học sinh như thể hiện trong Hình 8.3.

```
C:\MyApps>curl http://localhost:3000/students  
[{"id":1,"name":"David Miller","age":12,"grade":"A"  
, {"id":2,"name":"Adam Smith","age":13,"grade":"B"  
, {"id":3,"name":"John Willams","age":12,"grade":"A"  
}]
```

Hình 8.3: Thông tin của tất cả sinh viên

16. Để có được hồ sơ của một sinh viên cụ thể bằng cách chỉ định ID, tại Dấu nhắc lệnh, hãy chạy lệnh như sau:

```
curl http://localhost:3000/students/2
```

Lệnh này chỉ định ID là 2. Thông tin chi tiết về học sinh có ID là 2 được hiển thị như trong Hình 8.4.

```
C:\MyApps>curl http://localhost:3000/students/2  
{ "id":2, "name": "Adam Smith", "age":13, "grade": "B" }
```

Hình 8.4: Thông tin sinh viên có ID xác định

17. Để tạo một hồ sơ học sinh mới, tại dấu nhắc lệnh, hãy chạy lệnh như sau:

```
curl -X POST -H "Loại nội dung: ứng dụng/json" -d '{"tên\":"Michael Faraday\","tuổi\": 11, "lớp\":"A\"}' http://localhost:3000/students
```

Trong ví dụ này, yêu cầu POST được sử dụng để tạo một bản ghi sinh viên mới, với thông tin như sau:

- Tên: Michael Faraday
- Tuổi: 11
- Điểm: A
- Mã số: 4

Lệnh này tạo hồ sơ học sinh mới và hiển thị thông tin chi tiết như trong Hình 8.5.

```
C:\MyApps>curl -X POST -H "Content-Type: application/json" -d '{"name\":"Michael Faraday\","age\": 11, "grade\":"A\"}' http://localhost:3000/students
{"id":4,"name":"Michael Faraday","age":11,"grade":"A"}
```

Hình 8.5: Thông tin sinh viên mới

18. Để kiểm tra xem hồ sơ sinh viên này đã được tạo trong cơ sở dữ liệu hay chưa, tại Dấu nhắc lệnh, hãy chạy lệnh như sau:

```
curl http://localhost:3000/students
```

Lệnh này thực thi và hiển thị hồ sơ của tất cả học sinh như thể hiện trong Hình 8.6.

```
C:\MyApps>curl http://localhost:3000/students
[{"id":1,"name":"David Miller","age":12,"grade":"A"}, {"id":2,"name":"Adam Smith","age":13,"grade":"B"}, {"id":3,"name":"John Willams","age":12,"grade":"A"}, {"id":4,"name":"Michael Faraday","age":11,"grade":"A"}]
```

Hình 8.6: Sinh viên mới được thêm vào cơ sở dữ liệu

19. Để cập nhật hồ sơ học sinh với một ID cụ thể, tại Dấu nhắc lệnh, hãy chạy lệnh như sau:

```
curl -X PUT -H "Loại nội dung: ứng dụng/json" -d '{"tên\": \"David Willams\", \"tuổi\": 11, \"lớp\": \"A\"}' http://localhost:3000/students/3
```

Trong ví dụ này, yêu cầu PUT được sử dụng để cập nhật thông tin của sinh viên có ID là 3. Trong yêu cầu này, tên của sinh viên được đổi từ John thành David và tuổi được đổi từ 12 thành 11. Lệnh được thực thi và thông tin sinh viên được cập nhật được hiển thị như trong Hình 8.7.

```
C:\MyApps>curl -X PUT -H "Content-Type: application/json" -d '{"name\": \"David Willams\", \"age\": 11, \"grade\": \"A\"}' http://localhost:3000/students/3
{"id":3,"name":"David Willams","age":11,"grade":"A"}
```

Hình 8.7: Cập nhật thông tin sinh viên

20. Để kiểm tra xem hồ sơ học sinh này đã được cập nhật trong cơ sở dữ liệu hay chưa, tại Dấu nhắc lệnh, hãy chạy lệnh như sau:

```
curl http://localhost:3000/students
```

Lệnh này thực thi và hiển thị hồ sơ của tất cả học sinh như thể hiện trong Hình 8.8.

```
C:\MyApps>curl http://localhost:3000/students
[{"id":1,"name":"David Miller","age":12,"grade":"A"}, {"id":2,"name":"Adam Smith","age":13,"grade":"B"}, {"id":3,"name":"David Willams","age":11,"grade":"A"}, {"id":4,"name":"Michael Faraday","age":11,"grade":"A"}]
```

Hình 8.8: Thông tin sinh viên được cập nhật trong cơ sở dữ liệu

21. Để xóa hồ sơ học sinh có ID cụ thể, tại Dấu nhắc lệnh, hãy chạy lệnh như sau:

```
curl -X XÓA http://localhost:3000/students/4
```

Trong ví dụ này, yêu cầu DELETE được sử dụng để xóa bản ghi sinh viên có ID là 4. Lệnh xóa bản ghi và hiển thị thông báo như trong Hình 8.9.

```
C:\MyApps>curl -X DELETE http://localhost:3000/students/4
{"message":"Student deleted"}
```

Hình 8.9: Xóa hồ sơ học sinh

22. Để kiểm tra xem hồ sơ học sinh đã bị xóa khỏi cơ sở dữ liệu hay chưa, tại dấu nhắc lệnh, hãy chạy lệnh như sau:

```
curl http://localhost:3000/students
```

Lệnh này thực thi và hiển thị hồ sơ của tất cả học sinh như thể hiện trong Hình 8.10.

```
C:\MyApps>curl http://localhost:3000/students
[{"id":1,"name":"David Miller","age":12,"grade":"A"},
{"id":2,"name":"Adam Smith","age":13,"grade":"B"},{
"id":3,"name":"David Willams","age":11,"grade":"A"}]
```

Hình 8.10: Hồ sơ sinh viên đã bị xóa khỏi cơ sở dữ liệu

8.7 Sử dụng API RESTful với Express.js

Bây giờ, hãy xem xét trường hợp người dùng muốn cập nhật cùng một ứng dụng sinh viên để xác thực dữ liệu đã nhập nhằm tạo một sinh viên mới. Việc này có thể được thực hiện bằng API RESTful. Trong trường hợp này, cần tạo một lược đồ và dữ liệu được truyền bởi máy khách phải được xác minh dựa trên lược đồ đó.

Để thực hiện việc này, người dùng phải thực hiện các bước sau:

1. Đảm bảo rằng Node.js và Express.js đã được cài đặt.
2. Để sử dụng lược đồ, hãy mở cửa sổ Dấu nhắc lệnh.
3. Điều hướng đến cùng thư mục nơi Express.js được cài đặt.
4. Để cài đặt gói lược đồ, tại dấu nhắc lệnh, hãy chạy lệnh như sau:

```
npm cài đặt json-schema express-jonschema
```

Lệnh này cài đặt gói như thể hiện trong Hình 8.11.

```
C:\MyApps>npm install json-schema express-jonschema
added 1 package, and audited 70 packages in 2s

11 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
```

Hình 8.11: Cài đặt gói Schema

5. Để cài đặt gói xác thực, tại Dấu nhắc lệnh, hãy chạy lệnh như sau:

```
npm cài đặt express-validator
```

Lệnh này cài đặt gói như thể hiện trong Hình 8.12.

```
C:\MyApps>npm install express-validator
added 3 packages, and audited 73 packages in 1s
11 packages are looking for funding
run 'npm fund' for details
found 0 vulnerabilities
```

Hình 8.12: Cài đặt gói Validator

6. Để tạo lược đồ, hãy mở VS Code hoặc tệp Notepad và nhập mã được cung cấp trong Đoạn mã 8.

Đoạn mã 8:

```
{
  "$schema": "https://json-schema.org/draft/2019-
09/sơ đồ",
  "$id": "studentSchema",
  "title": "Sinh viên",
  "description": "Hồ sơ sinh viên",
  "kiểu": "đối tượng",
  "của cải": {

    "nhận dạng":
    {
      "kiểu": "số nguyên",
      "description": "Mã định danh duy nhất cho học sinh",

      "bắt buộc": đúng
    },
    "tên": {

      "kiểu": "chuỗi",
      "description": "Họ và tên đầy đủ của học sinh",
      "bắt buộc": đúng
    },
    "tuổi": {

      "kiểu": "số nguyên",
```

```

    "description": "Tuổi của học sinh",
    "bắt buộc": đúng
  },
  "cấp": {
    "kiểu": "chuỗi",
    "description": "Cấp bậc của học sinh",
    "bắt buộc": đúng
  }
}
}

```

Trong sơ đồ này, các chi tiết liên quan đến sinh viên được xác định như trong Bảng 8.4.

Kiểu dữ liệu chi tiết bắt buộc		
nhận dạng	chuỗi số	ĐÚNG VẬY
tên	nguyên chuỗi	ĐÚNG VẬY
lớp	số nguyên tuổi	ĐÚNG VẬY
		ĐÚNG VẬY

Bảng 8.4: Định nghĩa lược đồ

- Lưu tệp dưới dạng studentSchema.json trong cùng thư mục với Express.js đã được cài đặt.
- Tạo một bản sao của tệp studentHTTPapi.js và đặt tên là studentRESTfulapi.js.
- Mở tệp studentRESTfulapi.js.
- Để nhập trình xác thực và tham chiếu đến tệp studentSchema.json, hãy cập nhật mã như được đưa ra trong Đoạn mã 9.

Đoạn mã 9:

```

const express = yêu cầu('express');
ứng dụng const = express();
cổng const = 3000;
const { kiểm tra, validationResult } = require('express-validator');

const studentSchema = yêu cầu ('./studentSchema.json');
ứng dụng.use(express.json());

```

Mã bổ sung trong Đoạn mã 9 chỉ định rằng trình xác thực express phải trích xuất các lỗi xác thực từ yêu cầu và lưu trữ nó. Ngoài ra, tệp studentSchema.json được nhập và lưu trữ trong studentSchema trong đối tượng validationResult.

sự vật.

11. Để cập nhật mã tạo sinh viên mới để dữ liệu có thể được xác thực, hãy cập nhật mã như được đưa ra trong Đoạn mã 10.

Đoạn mã 10:

```
/ Tạo một học sinh mới
ứng dụng.post('/học sinh', [
  // Sử dụng kiểm tra để xác thực
  kiểm tra('tên').isString(),
  kiểm tra('tuổi').isInt({min: 9, max: 13}),
  kiểm tra('grade').isString(),
], (yêu cầu, độ phân giải) => {
  const errors = validationResult(req);

  nếu (!errors.isEmpty()) {
    //Xử lý lỗi xác thực
    trả về res.status(400).json({ message: 'Lỗi trong dữ liệu đã nhập' });

  } khác {
    const { tên, tuổi, lớp } = req.body;
    const student = { id: nextStudentId++, tên, tuổi, lớp };

    students.push(học sinh);
    res.status(201).json(học sinh);
  }
});
```

Trong Đoạn mã 10, hàm kiểm tra được sử dụng để xác thực xem kiểu dữ liệu của các chi tiết được cung cấp có giống với kiểu dữ liệu trong Bảng 8.4 hay không. Kiểu dữ liệu được kiểm tra bằng các phương thức `isString` và `isInt`. Mã cũng kiểm tra xem độ tuổi được chỉ định có nằm trong khoảng từ 9 đến 13 hay không bằng cách chỉ định giá trị min. Thuộc tính là 9 và thuộc tính tối đa là 13. Nếu tất cả dữ liệu nhập vào đều vượt qua kiểm tra, học sinh mới sẽ được tạo và thông tin chi tiết về học sinh mới sẽ được hiển thị. Nếu không, thông báo lỗi "Lỗi trong dữ liệu nhập" sẽ được hiển thị.

12. Giữ nguyên phần mã còn lại và lưu tệp.
13. Để khởi động máy chủ, hãy mở cửa sổ Dấu nhắc lệnh.

14. Tại dấu nhắc lệnh, hãy chạy lệnh như sau:

```
nút studentRESTfulapi.js
```

Lệnh này khởi động máy chủ. Khi máy chủ đã hoạt động, một thông báo sẽ được ghi vào bảng điều khiển cho biết máy chủ đang lắng nghe trên cổng được chỉ định, như minh họa trong Hình 8.13.

```
C:\MyApps>node studentRESTfulapi.js
Server is running on port 3000
```

Hình 8.13: Khởi động máy chủ

15. Để tạo hồ sơ học sinh mới, tại Dấu nhắc lệnh, hãy chạy lệnh như sau:

```
curl -X POST -H "Loại nội dung: ứng dụng/json" -d '{"tên": "Michael Faraday",
\tuổi": 11, \lớp":
"A"}' http://localhost:3000/students
```

Dữ liệu được cung cấp trong lệnh này vượt qua quy tắc xác thực. Do đó, học sinh được thêm vào cơ sở dữ liệu, như thể hiện trong Hình 8.14.

```
C:\MyApps>curl -X POST -H "Content-Type: application/json" -d
{"name": "Michael Faraday", "age": 11, "grade": "A"}
http://localhost:3000/students
{"id":4,"name":"Michael Faraday","age":11,"grade":"A"}
```

Hình 8.14: Thêm học sinh mới

16. Để kiểm tra xem hồ sơ học sinh này đã được tạo trong cơ sở dữ liệu hay chưa, tại Dấu nhắc lệnh, hãy chạy lệnh như sau:

```
curl http://localhost:3000/students
```

Lệnh này thực thi và hiển thị hồ sơ của tất cả học sinh như thể hiện trong Hình 8.15.

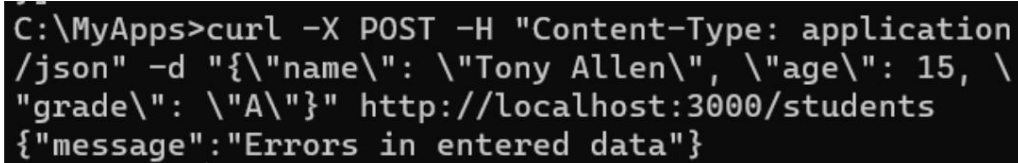
```
C:\MyApps>curl http://localhost:3000/students
[{"id":1,"name":"David Miller","age":12,"grade":"A"},
{"id":2,"name":"Adam Smith","age":13,"grade":"B"},{
"id":3,"name":"John Williams","age":12,"grade":"A"},{
"id":4,"name":"Michael Faraday","age":11,"grade":"A"
}]
```

Hình 8.15: Sinh viên mới được thêm vào cơ sở dữ liệu

17. Để tạo một bản ghi học sinh mới khác, tại Dấu nhắc lệnh, hãy chạy lệnh như sau:

```
curl -X POST -H "Content-Type: application/json" -d "{\"name\": \"Tony Allen\", \"age\": 15, \"grade\": \"A\"}" http://localhost:3000/students
```

Trong lệnh này, độ tuổi được chỉ định là 15, lớn hơn độ tuổi tối đa là 13. Do đó, dữ liệu này không vượt qua được xác thực và một thông báo lỗi sẽ hiển thị, như thể hiện trong Hình 8.16.



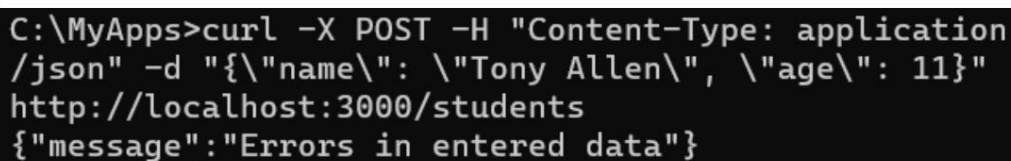
```
C:\MyApps>curl -X POST -H "Content-Type: application /json" -d "{\"name\": \"Tony Allen\", \"age\": 15, \"grade\": \"A\"}" http://localhost:3000/students {"message": "Errors in entered data"}
```

Hình 8.15: Thông báo lỗi hiển thị do độ tuổi không chính xác

18. Để tạo một bản ghi học sinh mới khác, tại Dấu nhắc lệnh, hãy chạy lệnh như sau:

```
curl -X POST -H "Loại nội dung: application/json" -d "{\"tên\": \"Tony Allen\", \"tuổi\": 11}" http://localhost:3000/students
```

Trong lệnh này, điểm không được chỉ định. Tuy nhiên, theo lược đồ, điểm là chi tiết bắt buộc. Do đó, dữ liệu này không vượt qua được xác thực và thông báo lỗi sẽ hiển thị, như minh họa trong Hình 8.17.



```
C:\MyApps>curl -X POST -H "Content-Type: application /json" -d "{\"name\": \"Tony Allen\", \"age\": 11}" http://localhost:3000/students {"message": "Errors in entered data"}
```

Hình 8.17: Thông báo lỗi hiển thị do thiếu điểm

8.8 Tóm tắt

API dịch hướng dẫn của người dùng sang định dạng có thể được hệ thống diễn giải để xử lý yêu cầu và gửi lại phản hồi.

API tạo điều kiện thuận lợi cho việc giao tiếp và tương tác giữa phần mềm các ứng dụng thông qua một bộ quy tắc và giao thức.

Tùy thuộc vào phạm vi và cách sử dụng, Node.js hỗ trợ một số API, chẳng hạn như API cốt lõi, API của bên thứ ba, API tùy chỉnh và API RESTful.

API HTTP sử dụng giao thức HTTP để tạo điều kiện thuận lợi cho việc giao tiếp và trao đổi dữ liệu giữa các hệ thống.

API RESTful là API HTTP tuân theo các nguyên tắc và hạn chế của phong cách kiến trúc REST để thiết kế các ứng dụng mạng.

Kiểm tra kiến thức của bạn

1. Bạn đã tạo một ứng dụng web đơn giản cho một thư viện. Bạn muốn tìm chi tiết của một cuốn sách bằng tên của nó. Hoàn thành mã:

```
app.get('/books/:name', (req, res) => { const bookName
    = req.params.name;

    // Thêm mã bị thiếu

    if (book)
        { res.json(book); } else

        { res.status(404).json({ message: 'Không tìm thấy sách' });

    } });

a) const book = books.find((b) => b.name.toLowerCase() b) const bookName
    = req.params.name;
    const book = books.find((b) => b.name.toLowerCase() ===
    bookName.toLowerCase()); c)
books.find(bookName) d) const
bookName = req.params.name.find(Name)
```

2. Bạn đang làm việc trên một ứng dụng web cần tương tác với máy chủ để truy xuất thông tin. Kiến trúc nào sẽ hữu ích cho việc thiết kế API nhằm đảm bảo tính đơn giản, khả năng mở rộng và tính không trạng thái?

- a) API RESTful
- b) API tùy chỉnh
- c) API định tuyến
- d) API tuần tự

3. Bạn đang làm việc trên một ứng dụng web. Khi giao tiếp với máy chủ bằng API HTTP, bạn sẽ sử dụng phương pháp nào để cập nhật thông tin hiện có trên máy chủ?

- a) GET
- b) POST
- c) DELETE
- d) PUT

4. Bạn chịu trách nhiệm duy trì đơn đăng ký khách sạn trực tuyến. Khách sạn có một bữa tiệc và người quản lý đã nhập sai tên khách. Bạn được yêu cầu cập nhật danh sách khách với tên chính xác.

Bạn sẽ sử dụng lệnh nào sau đây để thực hiện điều này?

- a) Cập nhật truy vấn
- b) `curl -X PUT -H "Content-Type: application/json" -d '{"key1": "value1", "key2": "value2"}' http://your-api-endpoint`
- c) `curl -X POST "Loại nội dung: ứng dụng/json" -d '{"key1": "value1", "key2": "value2"}' http://your-api-endpoint`
- d) Truy vấn XÓA

5. Bạn sẽ sử dụng phương thức HTTP nào sau đây để xóa tài nguyên khỏi máy chủ?

- a) NHẬN
- b) BÀI ĐĂNG
- c) XÓA
- d) ĐẶT

Câu trả lời để kiểm tra kiến thức của bạn

1 b	
2	Một t
3	ngày
4b	
5	c

Hãy tự mình thử nhé

1. Tạo một API RESTful đơn giản để quản lý thông tin chi tiết của nhân viên bằng Node.js và Express.

Bảng 8.5 liệt kê dữ liệu mẫu cho nhân viên.

Thứ tự	Tên	Tuổi	Chỉ định
1	Annie	27	Nhà phát triển
2	Jack	30	Nhà phân tích
3	Phao-lô	31	Nhà phát triển
4	Harry	29	Chuyên viên phân tích cao cấp

Bảng 8.5: Dữ liệu nhân viên

Cài đặt framework Express.js. Viết chương trình Node.js để tạo API quản lý thông tin nhân viên.

- a) Tạo lược đồ cho thông tin nhân viên.
- b) Tạo một mảng sử dụng dữ liệu trong Bảng 8.5.
- c) Xác định tuyến GET để lấy tất cả thông tin chi tiết của nhân viên.
- d) Xác định tuyến GET để lấy thông tin chi tiết của một nhân viên cụ thể bằng nhân viên.
- e) Thêm thông tin chi tiết của nhân viên mới.
- f) Xác thực dữ liệu đã nhập.
- g) Cập nhật nhân viên theo ID.
- h) Xóa nhân viên theo ID.
- i) Hiển thị kết quả trên bảng điều khiển.
- j) Lọc dữ liệu nhân viên dựa trên Chức danh được chỉ định từ máy chủ khi thực hiện yêu cầu GET bằng phương pháp tùy chỉnh.